This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

SynScale: Spatiotemporal Collaborative Autoscaling for Microservices in Edge-Clouds

Haogang Tong, Chunyang Meng, Shijie Song, Maolin Pan, Yang Yu*

***Abstract*—Edge-cloud environments constitute a heterogeneous computing paradigm that integrates resource-constrained edge servers with high-performance cloud servers. While microservices have revolutionized large-scale applications development by enhancing scalability and flexibility, achieving efficient microservice autoscaling—the dynamic adjustment of instances to maintain quality of service (QoS) and meet service-level agreements (SLA) targets—remains challenging in such environments. Most existing autoscaling techniques rely on metric forecasting and centralized or per-node control, causing them to overlook temporal evolution and server relationships and resulting in unstable and weakly coordinated scaling. To address these challenges, we present SynScale, a distributed collaborative autoscaling framework that strengthens both temporal sensitivity and structural coordination. SynScale introduces a spatiotemporal representation module that couples a temporal attention network with a multi-graph convolutional model. The temporal component distills behavioral trends from recent observations to ensure coherent responses to time-varying dynamics, while the spatial component performs relational reasoning over explicitly modeled inter-server correlations to yield structure-aware representations that support effective cross-node coordination. These spatiotemporal embeddings are then used within a multi-agent reinforcement learning paradigm, enabling distributed agents to generate context-aware scaling decisions that align local adaptability with system-wide efficiency. Experimental evaluations against state-of-the-art autoscaling techniques show that SynScale reduces average response time by 59.32%, SLA violations by 85.71%, and P95 latency by 73.2%, while also lowering resource cost. It further improves scaling stability—reflected by lower instance-time and longer instance lifetimes—and maintains lightweight, scale-stable runtime overhead, ensuring practical deployability in heterogeneous edge-cloud environments.**

***Index Terms*—Microservice Autoscaling, Edge-cloud Environment, Quality of Service, Service-Level Agreements, Graph Convolutional Network, Multi-Agent Reinforcement Learning.**

I. Introduction

In recent years, the proliferation of Internet of Things (IoT) devices and latency-sensitive applications has precipitated a paradigm shift in distributed computing architectures [1]. Traditional cloud computing models, while offering substantial computational power, often fall short in meeting the stringent latency requirements imposed by these emerging use cases [2]. This limitation has catalyzed the ascendance of edge computing as a complementary paradigm. Edge computing, by bringing computational resources closer to data sources and end-users, presents a compelling solution to the inherent limitations of centralized cloud architectures. The synthesis of these approaches, known as edge-cloud computing, strategically leverages the strengths of both paradigms, offering a hybrid solution that balances the proximity and real-time processing capabilities of edge devices with the scalability and extensive resources of cloud infrastructure.

Concurrently, the field of software engineering has witnessed a significant architectural evolution with the rise of microservices. This approach deconstructs monolithic applications into smaller, independently deployable services, offering enhanced modularity, scalability, and maintainability [3]. The adoption of microservices has been facilitated by advancements in containerization technologies such as Docker, orchestration platforms like Kubernetes, and serverless computing frameworks. These technologies have collectively redefined the landscape of software deployment and management. The convergence of edge-cloud computing and microservices architectures has given rise to novel deployment strategies, where microservice instances are distributed across the edge-cloud continuum [4], [5]. This approach aims to optimize request response times by leveraging on-site processing capabilities.

To accommodate the fluctuating request arrival rates in edge-cloud environments, the dynamic deployment of microservice instances across heterogeneous servers (i.e., servers with varying computational, memory, and storage capacities) is essential. This process, known as microservice autoscaling, adapts to the time-varying workload demands. However, the challenges posed by geographically uneven traffic patterns and the inherent resource heterogeneity across servers make the application of traditional autoscaling methods increasingly inadequate. The following analysis delves into these challenges in more detail.

Challenge 1: Scaling Under Spatiotemporal Dynamics and System Heterogeneity. Edge-cloud environments are inherently dynamic and heterogeneous. On one hand, user mobility and regional activity patterns introduce spatiotemporal workload fluctuations that result in server-specific and time-varying resource demands. On the other hand, the diverse resource capacities and deployment conditions across edge and cloud servers further compound this complexity, creating a highly non-uniform operational landscape. To substantiate this, we analyze the Shanghai Telecom dataset [6]–[8], which includes millions of records from over 9,000 mobile users across 3,000+ base stations. Our analysis reveals clear temporal variations and geographical disparities in request intensity, confirming that each server faces unique, evolving load patterns.

Haogang Tong, Chunyang Meng, Shijie Song are with the School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, China (email:{tonghg,mengchy3,songshj6}@mail2.sysu.edu.cn). Yang Yu and Maolin Pan are professors at the School of Software Engineering, Sun Yat-sen University, Zhuhai, China (email:{panml, yuy}@mail.sysu.edu.cn).

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

Most existing autoscaling approaches [9]–[11] focus on forecasting individual metrics such as workload or resource utilization to trigger proactive scaling. While effective for short-term adjustments, these prediction-driven designs ignore the evolving temporal interplay between system states and scaling decisions, leading to context-insensitive and unstable adaptation. They also remain heavily data-dependent, requiring large volumes of accurate training data to generate reliable predictions, and their forecast-oriented modeling fails to capture the multi-faceted dynamics of autoscaling. As a result, these limitations confine autoscalers to narrow, prediction-oriented adaptation, preventing them from leveraging historical cues and system-wide dynamics to make trend-aware and temporally consistent decisions under continuously changing conditions. Meanwhile, traditional rule-based approaches [12]–[14] are inherently inflexible—either relying on globally defined thresholds that ignore local conditions, or requiring per-server configurations that are labor-intensive and poorly coordinated. Even reinforcement learning (RL)-based methods [15]–[18] often adopt monolithic control structures that encode system-wide states but lack the granularity to reflect individual server dynamics. As a result, these approaches struggle to deliver fine-grained, context-aware scaling decisions under dynamic, heterogeneous conditions.

Challenge 2: Lack of Coordination Across Distributed Resources. Beyond the need for local adaptability, effective autoscaling in edge-cloud environments also demands coordination across geographically distributed and functionally interdependent servers. Microservices are often deployed as collaborative service chains, where multiple instances interact across servers to fulfill a single request. Their performance is thus not solely determined by individual server capacity, but by the efficiency of inter-server cooperation—including proximity, placement, and traffic migration.

However, most existing methods [15], [16], [18] treat servers as independent units and rely solely on local performance metrics such as CPU or memory usage. These metrics provide no insight into the structural relationships between servers, such as functional similarity, communication affinity, or migration tendencies. Consequently, scaling decisions are made in isolation, leading to redundant actions, cross-server bottlenecks, and underutilized synergy potential. Without explicit modeling and utilization of inter-server correlations, current approaches fail to achieve collaborative efficiency in distributed autoscaling.

Contribution. To address the above limitations and realize coordinated, temporally consistent autoscaling, we make the following contributions:

- We establish a structured modeling approach for diverse inter-server relationships by explicitly defining heterogeneous correlations across multiple resource attributes. This enables a more precise and adaptive representation of inter-server dependencies, facilitating informed decision-making and cooperative resource allocation in distributed environments.
- We design a spatiotemporal representation learning mechanism that integrates a temporal attention network with multi-graph convolution. The temporal component distills trend-aware contextual representations from historical observations, enabling the policy to perceive evolving behavioral patterns and make temporally consistent scaling decisions, while the spatial component performs graph-based relational reasoning over explicitly modeled inter-server relationships to yield structure-aware representations that enhance distributed coordination.
- We develop a distributed collaborative autoscaling framework in which multiple agents jointly optimize scaling policies to balance local adaptability and global QoS objectives. This design effectively leverages system heterogeneity and improves computational efficiency and scalability.
- We construct realistic, geographically distributed workloads using real-world data and conduct comprehensive empirical evaluations against state-of-the-art methods. Our results demonstrate substantial improvements in response time, SLA adherence, and resource efficiency, reinforcing the effectiveness and scalability of our approach.

II. Related work

A. Related Work

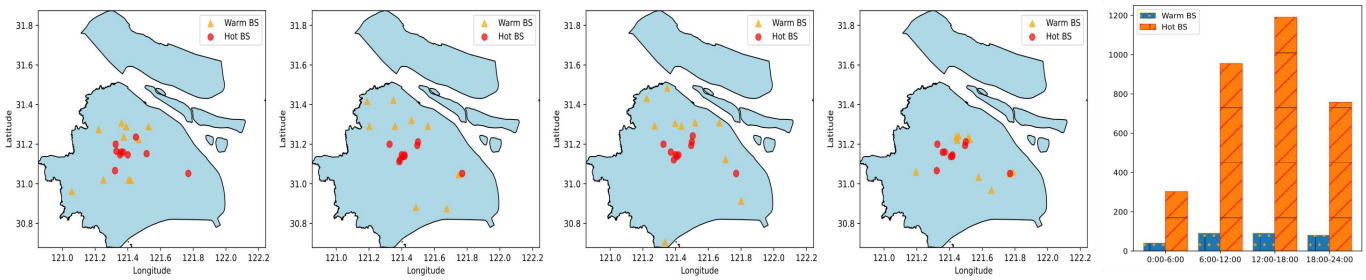
The domain of microservice autoscaling has witnessed significant research momentum in recent years. This section presents a comprehensive review of microservice autoscaling studies, categorized into distinct methodological approaches.

Rule-based autoscaling methods [19]–[29] rely on predefined thresholds for scaling decisions based on monitored resource metrics. These methods are favored for their simplicity and low computational overhead, making them suitable for resource-constrained environments. However, they are inherently limited by their static nature, often failing to adapt to dynamic and unpredictable environments. Predictive autoscaling methods [9]–[11], [30]–[33] attempt to mitigate the reactivity limitations of rule-based methods by forecasting future resource demands based on historical data. While these methods can reduce scaling latency and improve performance, they are data-dependent, requiring large volumes of accurate training data to generate reliable predictions. Furthermore, their time modeling remains inherently forecast-oriented—treating temporal information merely as input features for value prediction—thus failing to capture the evolving behavioral context that links past system changes with scaling behavior. As a result, they lack temporal awareness in decision making, often leading to inconsistent or short-sighted scaling under continuously changing conditions. Both rule-based and predictive methods fail to leverage inter-server dependencies, making them unsuitable for collaborative scaling across distributed environments.

Machine learning (ML)-based methods [28], [34]–[39] have been employed to dynamically adjust scaling actions by learning from historical data and identifying patterns in resource utilization. These methods can adapt to a broader range of scenarios, but they are heavily reliant on large datasets and accurate labeling, which can be difficult to obtain in complex, heterogeneous systems. Moreover, the generalization capabilities of these models are often limited, especially

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566



(a) 0:00-6:00 (b) 6:00-12:00 (c) 12:00-18:00 (d) 18:00-24:00 (e) the total volume of requests

Fig. 1: Time-Varying and Geographically Unevenly Distributed Workload

when encountering novel environments or workload shifts not present in the training data. Despite these challenges, ML-based methods have shown promise in offering scenario-specific autoscaling solutions tailored to observed conditions.

Reinforcement learning (RL)-based approaches [16]–[18], [40]–[46] represent a significant departure from rule-based and predictive methods by continuously learning and optimizing policies through interactions with the environment. These methods are well-suited for dynamic environments where data is sparse and workloads are highly variable. However, monolithic policy models used in most RL approaches tend to overlook the heterogeneity of individual servers and inter-server synergies, often leading to inefficient scaling actions and increased computational overhead.

Recently, a few studies have explored the integration of graph convolutional networks (GCNs) into RL-based autoscaling to capture service dependencies [15], [47]. These approaches enhance the feature representation of system states by incorporating structural information among microservices. While promising, such methods typically assume static service interaction graphs or topology-based adjacency, and often fail to model server-level collaborative relationships explicitly, limiting their ability to fully exploit cross-server synergies in heterogeneous environments.

B. Key Insights

Through comprehensive experiments (Sec. V), we further analyze and reveal the limitations of existing approaches regarding the challenges previously outlined. To clearly distinguish our approach from these methods, we summarize our key insights as follows.

Insight 1: One Policy Does Not Fit All—Rethinking Autoscaling for Heterogeneous Edge Servers. In edge-cloud environments, geographically distributed servers experience highly dynamic and server-specific conditions. Analysis of the Shanghai Telecom trace [6]–[8]—millions of requests from 9 000+ users across 3 000+ base stations (BSs) over six months—shows that, after partitioning each day into four equal intervals, the spatial distribution of hot (high-load) and warm (low-load) BSs shifts markedly (Fig.1a-d). Both request volume (Fig.1e) and geographical concentration fluctuate over time, leading to evolving and asymmetric scaling demands across servers.

Existing autoscaling techniques do not satisfy the dual requirement of *local adaptability* and *global coordination*. (i) *Rule- and prediction-based methods* [9], [48], [49] typically operate on statically configured, globally tuned thresholds (e.g., CPU utilization or queue length). Such one-size-fits-all heuristics lack the flexibility to accommodate server-specific variations, resulting in limited contextual precision and reduced responsiveness in dynamic environments. (ii) *Centralized RL approaches* [16], [17], [40], [50] encode system-wide information into a monolithic global state representation and learn a unified policy for all servers. This design (a) suffers from scalability issues due to exponential state-space growth as server count increases, (b) tightly couples the learned policy to a fixed environment size, and (c) fails to account for heterogeneity across servers, leading to uniform decisions that are ill-suited for distributed, diverse conditions. Moreover, the lack of an explicit collaboration mechanism prevents such methods from exploiting the complementary capacities of edge nodes.

These limitations motivate a shift toward decentralized yet collaborative autoscaling—exemplified by our proposed SynScale—which learns server-specific scaling policies that are locally adaptive while supporting lightweight coordination across the system.

Insight 2: Beyond Isolated Scaling: Graph-Based Relational Modeling for Collaborative Autoscaling. A fundamental limitation of existing autoscaling methods is their narrow-scope decision-making, which primarily relies on per-server resource utilization metrics while neglecting the structured relationships among servers. Many traditional methods, such as rule-based autoscalers (e.g., Kubernetes Horizontal Pod Autoscaler (HPA) [51]), make scaling decisions based solely on predefined thresholds, treating each server in isolation without considering cross-server interactions. Meanwhile, reinforcement learning (RL)-based approaches [18], [40] optimize scaling policies through global decision-making, encoding system states in a monolithic fashion. While differing in their decision-making scope, both approaches fail to explicitly model inter-server relationships, resulting in uncoordinated scaling actions and inefficient resource utilization. Without a structured understanding of how servers influence each other, these methods suffer from inefficient resource allocation, increased scaling overhead, and degraded application performance.

A more effective autoscaling mechanism must therefore move beyond isolated or monolithic decision-making toward a relational perspective—one that captures how servers interact,

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

complement each other, and collectively shape system behavior. Incorporating such relational structure enables autoscalers to reason collaboratively across distributed nodes rather than reacting to local signals alone. This perspective forms the conceptual foundation for our subsequent design, where we leverage structured inter-server relationships to support coordinated and scalable autoscaling decisions.

III. Problem Formulation and System Design

A. System Model

We consider a distributed edge-cloud environment where user requests are served by microservices deployed across geographically distributed, resource-heterogeneous servers. Each microservice may run multiple instances, and a request may invoke a dependency chain spanning several servers. The key notations used throughout this paper are summarized in Table I.

Heterogeneity in such environments arises from both the network and device layers [52]. Network heterogeneity appears in two forms [53]: edge-to-edge variations in bandwidth and propagation delay among edge nodes, and edge-to-cloud discrepancies between low-latency intra-edge local area networks (LANs) and the higher-latency backhaul links to the cloud. Device heterogeneity comprises variations in both resource capacities (e.g., CPU, memory, storage) and computational capabilities, the latter reflected in differences in processing speeds caused by variations in hardware architecture, operating systems, and system-level optimizations [54].

The edge-cloud environment is modeled as a Directed Acyclic Graph (DAG), denoted as $G_E = (\mathcal{V}, \mathbb{A})$, where $\mathcal{V}$ is the server set which consists of a set of heterogeneous edge devices and a cloud data center, and $\mathbb{A} = \{\mathbf{A}_{\text{neigh}}, \mathbf{A}_{\text{poi}}, \mathbf{T}\}$ is the set of inter-server correlations introduced in this paper. The resource capabilities of each server $v_i \in \mathcal{V}$ are quantified in terms of CPU, memory, storage volume, denoted as $\mathcal{W}_i^{cpu}$, $\mathcal{W}_i^{mem}$, $\mathcal{W}_i^{disk}$, respectively.

TABLE I: Notations and Descriptions

Symbol	Description
$\mathcal{V}$	Set of servers
$\mathbb{A}$	Set of adjacency matrices for server correlations
$\mathcal{W}_i^{cpu}$	Number of CPU cores on servers
$\mathcal{W}_i^{mem}$	Memory capacity of servers
$\mathcal{W}_i^{disk}$	Storage volume of servers
$\mathcal{M}$	Set of microservices
$\mathcal{E}$	Call relationships among microservices
$\triangle_{m,m'}$	The amount of data transferred from microservice m to m'
$\mathcal{F}$	The functionality set of the considered application
w_i^{cpu}	Number of CPU cores required to deploy service i
w_i^{mem}	Amount of memory required to deploy service i
w_i^{disk}	Amount of storage required to deploy service i
$\alpha_m(t)$	Workload of each instance of service m
λ_m	Processing rate of an instance of service m

1) Correlations of Servers: To capture the inter-server synergies in edge-cloud environments, we propose and formalize three types of server correlations: neighborhood, functional similarity, and traffic connectivity. These correlations are vital for optimizing collaborative autoscaling decisions by considering the structural dependencies between servers.

Neighborhood. Spatial proximity is a critical factor in determining which servers are likely to benefit from collaborative scaling. To model this, we define a binary adjacency matrix $\mathbf{A}_{\text{neigh}} = [a_{ij}]_{i,j=1}^{|\mathcal{V}|}$, where each element a_{ij} indicates whether server v_i and server v_j are geographically adjacent:

$$a_{ij} = \begin{cases} 1, & \text{if } v_i \text{ and } v_j \text{ are adjacent} \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

This matrix captures the spatial layout of the system, revealing which server pairs are in close geographic proximity. Such relationships are particularly valuable in edge-cloud scenarios, where servers within the same proximity can coordinate their scaling actions more effectively, reducing communication overhead and end-to-end latency. Proximity-based collaboration ensures that scaling decisions are context-aware and responsive to physical deployment characteristics, thereby improving real-time performance.

Functional similarity. Besides proximity, the functional suitability of servers for specific workloads is also critical. Different servers may be optimized for different types of tasks, such as compute-intensive or memory-bound tasks. To quantify the functional similarity between servers, we define a Point of Interest (POI) vector for each server, which includes attributes such as its geographical location, resource availability (CPU cores, memory, storage), and current load metrics. The POI vector is updated continuously at runtime to reflect the dynamic state of the server. The cosine similarity between two servers' POI vectors is calculated to measure their functional similarity:

$$\tilde{a}_{ij} = \frac{\mathbf{p}_i \cdot \mathbf{p}_j}{\|\mathbf{p}_i\| \|\mathbf{p}_j\|} \tag{2}$$

where $\mathbf{p}_i$ and $\mathbf{p}_j$ are the POI vectors for servers v_i and v_j, respectively. Since the POI features are non-negative, $\tilde{a}_{ij}$ lies within $[0, 1]$. To construct an adjacency matrix applicable to graph convolution, we further binarize the similarity matrix using a threshold τ_{poi}:

$$a_{ij} = \begin{cases} 1, & \text{if } \tilde{a}_{ij} \geq \tau_{\text{poi}}, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

We collect these pairwise similarities into a matrix $\mathbf{A}_{\text{poi}} = [a_{ij}]_{i,j=1}^{|\mathcal{V}|}$, which represents the functional similarity adjacency among all servers. This measure reflects the similarity in operational characteristics between servers and is updated continuously to track state changes. Incorporating functional similarity helps align scaling decisions with each server's capabilities and current conditions. Here, the threshold is set to $\tau_{\text{poi}} = 0.6$ in our implementation.

Traffic connectivity. User mobility across servers is a key factor influencing workload distribution in edge-cloud environments. As users move between servers over time, they form dynamic patterns of spatial and temporal flows. To capture these spatiotemporal relationships, we define a traffic connectivity matrix $\mathbf{T}$, which tracks whether there is significant traffic flow between two servers. If the traffic flow

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

$\eth_{ij}$ between servers v_i and v_j exceeds a threshold $\tau = 20$, the connection between them is considered substantial:

$$\mathbf{T}_{ij} = \begin{cases} 1, & \text{if } \eth_{ij} \geq \tau \\ 0, & \text{if } \eth_{ij} < \tau \end{cases} \tag{4}$$

This matrix helps model traffic dependencies between servers, identifying pairs of servers that frequently experience traffic flows from each other. By identifying these connections, the autoscaling algorithm can proactively prepare instances on servers with strong traffic connectivity, ensuring that they are ready to handle the incoming demand and minimizing latency.

2) Microservice-Based Application: Modern applications are increasingly structured as microservices to enhance modularity, scalability, and independent deployment. We formally represent a microservice-based application as a directed acyclic graph (DAG), denoted by $G_M = (\mathcal{M}, \mathcal{E})$, where $\mathcal{M}$ is the set of microservices (nodes), and $\mathcal{E} \subseteq \mathcal{M} \times \mathcal{M}$ denotes the set of directed inter-service dependencies (edges). For any two microservices $m, m' \in \mathcal{M}$, a directed edge $\langle m, m' \rangle \in \mathcal{E}$ implies that m' depends on the output produced by m during execution. The parameter $\triangle_{m,m'}$ captures the volume of data transmitted from m to m' during execution.

Each application supports a set of functionalities, denoted by $\mathcal{F}$. Following prior work [13], we model each functionality $f_i \in \mathcal{F}$ as a sequence of microservices that must be executed in a predefined order, i.e., $f_i = \langle m_1, \ldots, m_{|f_i|} \rangle$, where $|f_i|$ denotes the number of microservices involved. This formalism enables the system to associate functionality-specific request flows with subgraphs of the overall microservice DAG.

Microservices are executed through their deployed instances, each corresponding to a lightweight container hosted on a server. The deployment of an instance consumes server resources including CPU, memory, and storage. For each microservice m, we denote its per-instance resource demands by w_m^{cpu}, w_m^{mem}, and w_m^{disk}, respectively.

3) Request Response Time: Users request different functionalities provided by the application, each typically composed of a sequence of interdependent microservices. We denote such a functionality by $f_i = \langle m_1, ..., m_{|f_i|} \rangle$, where m_j represents the j-th microservice in the chain. Given the geographic distribution of users in an edge-cloud environment, requests are first received by the base station nearest to the user, then routed to one or more nearby servers for processing. For the same functionality, requests may be served by different combinations of servers depending on location and load balancing, thus forming distinct processing paths.

We define a path $l = \langle v_1, ..., v_{|f_i|} \rangle$, where each v_j indicates the server executing microservice m_j of functionality f_i. The expected response time $T_{f_i}(t)$ for functionality f_i in time slot t is the weighted sum over all feasible paths:

$$T_{f_i}(t) = \sum_{v_1=1}^{|\mathcal{V}|} \ldots \sum_{v_{|f_i|}=1}^{|\mathcal{V}|} \mathrm{P}(f_i, l) T_{l,f_i}(t) \tag{5}$$

where $\mathrm{P}(f_i, l)$ is the probability that a request for f_i is processed along path l, and $T_{l,f_i}(t)$ is the corresponding response time along that path.

The routing probability is defined as:

$$\mathrm{P}(f_i, l) = \prod_{j=1}^{|f_i|} \mathrm{P}(m_j, v_j) \tag{6}$$

where $\mathrm{P}(m_j, v_j)$ denotes the probability that microservice m_j is handled by a server v_j. Assuming round-robin load balancing, this probability is proportional to the number of instances of m_j deployed on v_j:

$$\mathrm{P}(m, v_j) = \frac{n_{m,v_j}(t)}{\sum_{k=1}^{|\mathcal{V}|} n_{m,v_k}(t)} \tag{7}$$

where $n_{m,v_j}(t)$ is the number of m's instances on v_j.

The overall path-level response time $T_{l,f_i}(t)$ is composed of four components: transmission delay, propagation delay, queuing delay, and execution delay, formally expressed as:

$$T_{l,f_i}(t) = T_{l,f_i}^{trans}(t) + T_{l,f_i}^{prop}(t) + T_{f_i}^{queue}(t) + T_{f_i}^{exec}(t) \tag{8}$$

Each term captures a different stage of end-to-end request processing.

The transmission delay accounts for the time needed to transfer intermediate data between dependent microservices hosted on different servers. For two consecutive microservices $m_i \to m_j$ deployed on v_i and v_j respectively, the transmission delay is defined as:

$$T_{l,m_i,m_j}^{trans}(t) = \frac{\triangle_{m_i,m_j}}{\mathrm{B}(v_i, v_j)} \tag{9}$$

where $\triangle_{m_i,m_j}$ is the data volume to be transmitted and $\mathrm{B}(v_i, v_j)$ is the bandwidth between the servers. If $v_i = v_j$, the delay is considered negligible. Aggregating over the service chain, the total transmission delay becomes:

$$T_{l,f_i}^{trans}(t) = \sum_{j=1}^{|f_i|-1} T_{l,m_j,m_{j+1}}^{trans}(t) \tag{10}$$

Next, the propagation delay captures the time for signals to traverse the physical distance between servers. For each pair of consecutive servers v_j and v_{j+1} on the path, the propagation delay is denoted by $T_{v_j,v_{j+1},f_i}^{prop}(t)$, and the cumulative delay over the full path is:

$$T_{l,f_i}^{prop}(t) = \sum_{j=1}^{|l|-1} T_{v_j,v_{j+1},f_i}^{prop}(t) \tag{11}$$

Following data transmission and propagation, the request is enqueued and executed on the target instance. To capture this, we adopt the standard M/M/1 queuing model. For a microservice m, the combined queuing and execution delay is given by:

$$T_m^{queue}(t) + T_m^{exec}(t) = \underbrace{\frac{\alpha_m(t)}{\lambda_m(\lambda_m - \alpha_m(t))}}_{\text{queue}} + \underbrace{\frac{1}{(\lambda_m)}}_{\text{execution}} \tag{12}$$

where $\alpha_m(t)$ denotes the arrival rate of requests at the instance, and λ_m represents the processing rate, measured in requests

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

per second (RPS). Summing over all microservices along the functionality chain, we obtain:

$$T_{f_i}^{queue}(t) + T_{f_i}^{exec}(t) = \sum_{j=1}^{|f_i|} T_{m_j}^{queue}(t) + T_{m_j}^{exec}(t) \tag{13}$$

Finally, the overall expected application response time is obtained by aggregating over all functionalities, weighted by their request distribution:

$$T^{app}(t) = \sum_{i=1}^{|\mathcal{F}|} \mathrm{P}(f_i, t) T_{f_i}(t) \tag{14}$$

where $\mathrm{P}(f_i, t)$ denotes the probability of functionality f_i being requested at time t, calculated as:

$$\mathrm{P}(f_i, t) = \frac{\sum_{k=1}^{|\mathcal{V}|} \alpha_{f_i, v_k}(t)}{\sum_{j=1}^{|\mathcal{F}|} \sum_{k=1}^{|\mathcal{V}|} \alpha_{f_j, v_k}(t)} \tag{15}$$

with $\alpha_{f_i, v_k}(t)$ denoting the request volume for functionality f_i at server v_k.

B. Problem Formulation

The goal of microservice autoscaling in edge-cloud environments is to ensure application-level QoS, typically reflected by low response latency and stable system performance. Given the heterogeneity and resource constraints of distributed servers, we aim to minimize the average response time of the application by determining a time-varying deployment strategy for microservice instances while satisfying multi-dimensional resource constraints and ensuring request serviceability. Formally, the problem is defined as:

$$\min_{n_{m,v}(t)} \lim_{T \to \infty} \frac{1}{T} \sum_{t=1}^{T} T^{app}(t), \quad m \in \mathcal{M},\ v \in \mathcal{V},\ t \in \mathbb{Z}^+ \tag{16a}$$

$$\text{s.t.} \quad \sum_{v \in \mathcal{V}} n_{m,v}(t) \geq 1, \quad \forall m \in \mathcal{M} \tag{16b}$$

$$\sum_{m \in \mathcal{M}} n_{m,v}(t)\, w_m^r \leq \mathcal{W}_v^r, \quad \forall v \in \mathcal{V},\ \forall r \in \mathcal{R} \tag{16c}$$

$$\alpha_m(t) > \lambda_m, \quad \forall m \in \mathcal{M} \tag{16d}$$

In the above, $n_{m,v}(t)$ denotes the number of instances of service m deployed on server v at time t, and $\mathbb{Z}^+$ represents the set of positive integers. Equation 16a defines the objective of minimizing long-term average application response time $T^{app}(t)$ across all services and servers. Constraint 16b ensures service availability by requiring at least one instance of each microservice to be deployed. Constraint 16c compactly enforces the per-server limits on CPU, memory, and disk resources, where $r \in \mathcal{R}$ indexes these resource types and w_m^r and $\mathcal{W}_v^r$ denote the corresponding per-instance demand and server capacity. Finally, 16d ensures system stability by enforcing that the processing rate of each service exceeds its arrival rate.

IV. Algorithm Design

This section presents the design of SynScale, with the overall architecture illustrated in Fig. 2. The subsequent subsections provide detailed descriptions of its components and their underlying design principles.

A. Spatiotemporal Representation Module

This subsection introduces the spatiotemporal representation module, which combines a temporal attention mechanism with a multi-graph convolutional network based on Chebyshev convolution (ChebConv) [55]. As illustrated in parts B and A of Fig. 2, these components jointly process historical observations and multi-relational server graphs to produce spatiotemporal representations used in subsequent policy learning.

1) Temporal Attention Network: The input to this module is the historical observation sequence $X = x_1, ..., x_W$, where each x_t includes server states, executed actions, and application response at time t. We set $W = 5$ in our implementation. To extract informative temporal features, we adopt a temporal attention mechanism that learns to focus on key historical points by dynamically assigning attention weights along the temporal dimension:

$$\begin{cases} \mathrm{A}_{\text{att}} = \text{softmax}\left(\Theta \sigma\left(X_1 X_2 + b\right)\right) & (17) \\ X_1 = W_2 (X^T W_1) & (18) \\ X_2 = X^T W_3^T & (19) \end{cases}$$

Here, Θ, W_1, W_2, W_3 are learnable parameters, and σ is a nonlinear activation. The resulting attention weights A_{att} highlight temporally salient points that correlate scaling actions with their outcomes. The output H_{temp} is obtained by aggregating weighted inputs:

$$H_{\text{temp}} = \mathrm{A}_{\text{att}} X \tag{20}$$

This temporal encoding captures the dynamic evolution of system behavior and its correlation with past scaling actions, allowing the policy to internalize performance trends.

2) Multi-graph Network: The input to this module includes the current server states and a set of explicitly defined graphs $\mathbb{A} = \{\mathbf{A}_{\text{neigh}}, \mathbf{A}_{\text{poi}}, \mathbf{T}\}$ that capture spatial proximity, functional similarity, and traffic connectivity, respectively, as described in §III-A1. These graphs form a unified multi-relational server topology $G_E = (\mathcal{V}, \mathbb{A})$, where each server is a vertex.

We perform Chebyshev spectral convolution over each graph in parallel. The results are concatenated across the feature dimension to form an enriched representation that encodes multi-faceted inter-server relationships:

$$H^{(l)} = ||_{\tau=1}^{|\mathbb{A}|} \sigma\left(\sum_{k=0}^{K} T_k(\hat{A}_\tau) H^{(l-1)} \Theta^{(l)}\right) \tag{21}$$

where σ is the activation function (i.e., ReLU), $T_k(x)$ denotes the Chebyshev polynomials that are recursively defined as $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$, with $T_0(x) = 1$ and $T_1(x) = x$. K is the order of the Chebyshev convolution, which determines the range of neighbors (up to K-hop) considered during the convolution. $A_\tau \in \mathbb{A}$ represents the τ-th adjacency matrix in $\mathbb{A}$. $\hat{A}_\tau = \tilde{D}^{-\frac{1}{2}} \tilde{A}_\tau \tilde{D}^{-\frac{1}{2}}$ is the normalized

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

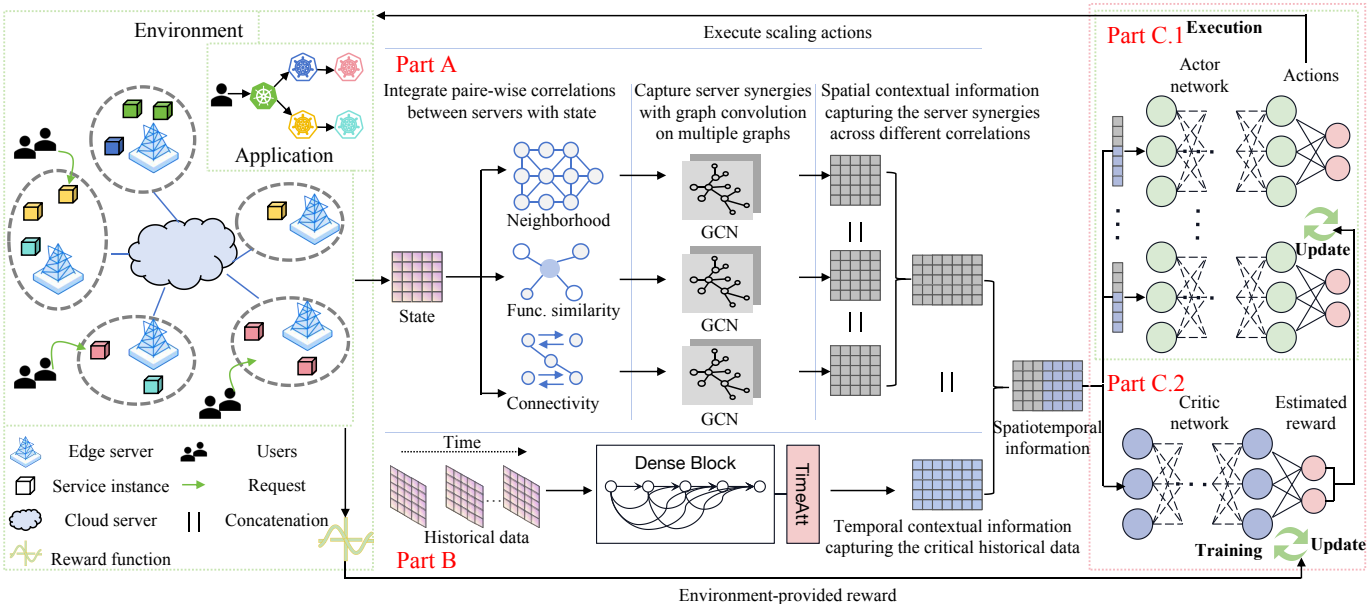


Fig. 2: Algorithm Overview

adjacent matrix, where $\tilde{A}_\tau = A_\tau + I$ and $\tilde{D}$ is a diagonal matrix with $\tilde{D}_{ii} = \sum_j \tilde{A}_{\tau_{ij}}$. $H^{(l-1)}$ represents the feature matrix from the previous layer, which for the first layer is the initial server states. $\Theta^{(l)}$ is the learnable parameters. $||$ denotes the concatenation operator. This process is repeated over two layers, and the final output H_{spat} captures server-aware features enriched with multi-relational context.

3) Spatiotemporal Fusion: The temporal and spatial embeddings are concatenated along the feature axis:

$$H_{\text{final}} = H_{\text{temp}} || H_{\text{spat}} \tag{22}$$

This spatiotemporal embedding is passed to the scaling policy network, enabling it to reason over both historical dynamics and structural synergies. The fusion of temporal attention and graph-based structural encoding ensures that the policy not only reacts to real-time conditions, but also aligns with the long-term collaborative potential of the edge-cloud system.

B. Distributed Collaborative Autoscaling

To realize adaptive and collaborative autoscaling in distributed environments, we instantiate our framework with Multi-Agent Proximal Policy Optimization (MAPPO) [56] under the Centralized Training and Decentralized Execution (CTDE) paradigm [57]. This setup allows agents deployed on each server to make scaling decisions independently while being trained cooperatively, enabling the system to integrate server-specific conditions with system-wide coordination.

SynScale trains two neural networks parameterized by θ and ϕ: an actor network π_θ for action selection and a value network V_ϕ for global state evaluation. Depending on the application, these networks can either be shared among all agents when their state–action spaces are homogeneous, or instantiated independently with arbitrary architectures when heterogeneous behavior is required.

For each server v_i, the actor π_i receives a server-centric observation. As shown in Fig. 3, this observation consists of two components. The first is the local extended state $\tilde{s}_i = (s_i, X_i)$, where s_i denotes the current state of server v_i—capturing its resource condition and workload-related characteristics—and X_i is its historical observation sequence used by the temporal attention module. The second component is the set of states from other servers, denoted as $\{s_j : j \in \mathcal{N}_i\}$, where $\mathcal{N}_i$ is the selected neighborhood used by the multi-graph convolution to construct spatial relational features. The receptive field size $k_i = |\mathcal{N}_i|$ can be flexibly configured depending on deployment needs. We denote the entire input compactly as $\mathbf{o}_i = (\tilde{s}_i, \{s_j : j \in \mathcal{N}_i\})$. As the policy operates in a distributed manner and its network design depends only on this per-server observation rather than the global system state, changes in the number of servers do not affect the dimensionality of s_i; only modifications to the local state space itself would require retraining. Given $\mathbf{o}_i$, the actor π_i produces a probability distribution $\pi_i(a \mid \mathbf{o}_i)$ over the discrete autoscaling actions in the action space $\mathcal{A} = \{-2, -1, 0, 1, 2\}$, where each action specifies a step-wise adjustment to the number of microservice instances on server v_i. The agent then selects the most probable action a_i for execution.

During centralized training, the value network V_ϕ receives a global representation of the system. Let $\mathbf{z}$ denote the global state constructed by concatenating the observations of all agents. Given $\mathbf{z}$, the critic outputs a scalar that estimates the expected long-term return associated with the current joint condition of the system. This global evaluation reflects how desirable the overall system state is and provides training-time guidance for improving the shared policy toward cooperative behavior.

Because autoscaling is a cooperative control task, all agents share the same reward signal. In this work, we adopt parameter sharing under a homogeneous state–action definition. Prior

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

studies [56], [58] have shown that sharing policy parameters across homogeneous agents can improve learning efficiency and overall policy performance. Consequently, each agent employs an actor and a critic that share the same parameter sets θ and ϕ. In addition, we configure the receptive field to be global, meaning that the actor's observation $\mathbf{o}_i$ consists of the local extended state $\tilde{s}_i$ together with the states of all other servers (i.e., $\{s_j : j \in \mathcal{V} \setminus \{i\}\}$).

Training. The training phase of the autoscaler involves jointly optimizing the policy function π_θ and the value function V_ϕ via stochastic gradient ascent. The policy is updated to improve decision quality and maintain exploration capacity, while the value function estimates expected returns to support advantage computation and cooperative learning. At each iteration, we minimize a composite loss:

$$L_t(\theta, \phi) = \hat{\mathbb{E}}_t[-L_t^{\text{CLIP}}(\theta) + c_1 L_t^{\text{VF}}(\phi) - c_2 S[\pi_\theta]] \quad (23)$$

where $\hat{\mathbb{E}}_t$ denotes the empirical average over a finite batch of samples, and c_1, c_2 are coefficient hyperparameters. The first term, $L_t^{\text{CLIP}}(\theta)$, represents the clipped surrogate objective, defined as:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t[\min(p_t(\theta)\hat{A}_t, \text{clip}(p_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)] \quad (24)$$

where $p_t(\theta) = \frac{\pi_\theta(a_t|\mathbf{o}_t)}{\pi_{\theta_{\text{old}}}(a_t|\mathbf{o}_t)}$ denotes the ratio of action probabilities which quantifies how the probability of selecting action a_t has changed between the old policy $\pi_{\theta_{\text{old}}}$ and the current policy π_θ based on the same state $\mathbf{o}_t$. The clipping operation, parameterized by ϵ, constrains the policy update, mitigating the risk of excessively large policy deviations that could lead to performance collapse. Here, $\hat{A}_t$ denotes the generalized advantage estimate (GAE) [59], a standard technique for reducing variance in policy gradient estimation. To further stabilize learning, we normalize the advantages using the PopArt method [60].

The second term in the composite loss, $L_t^{\text{VF}}(\phi)$, represents the squared-error loss for the value function:

$$L^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t\left[\left(V_\phi(\mathbf{z}_t) - y(t)\right)^2\right]. \quad (25)$$

The target value $y(t)$ is computed as $y(t) = r_t + \gamma V_\phi(\mathbf{z}_{t+1})$, and r_t is the environment reward given by the autoscaling reward function

$$r_t = \delta\,(\text{SLA} - res_t) - \beta \sum_{i=1}^{|\mathcal{M}|} n_i(t), \quad (26)$$

with SLA denoting the response-time threshold, res_t the measured application response time, and $n_i(t)$ the number of instances of microservice i. The coefficients are set to $\delta = 0.6$ and $\beta = 0.4$ to balance SLA compliance against resource consumption, and γ is the standard discount factor. This term trains the value network to approximate expected returns, supporting stable advantage estimation.

The final term, $S[\pi_\theta]$, is an entropy bonus that promotes exploration by maintaining policy stochasticity:

$$S[\pi_\theta](s_t) = -\sum_a \pi_\theta(a|\mathbf{o}_t) \log \pi_\theta(a|\mathbf{o}_t) \quad (27)$$

where a represents the possible actions that can be taken by the policy π_θ in state $\mathbf{o}_t$.

Update. The policy and value function are updated iteratively via stochastic gradient ascent:

$$\text{policy:} \quad \theta \leftarrow \theta + \eta_\pi(\nabla_\theta L_t^{\text{CLIP}}(\theta) + c_2 \nabla_\theta S[\pi_\theta]), \quad (28)$$

$$\text{value:} \quad \phi \leftarrow \phi - \eta_v c_1 \nabla_\phi L_t^{\text{VF}}(\phi) \quad (29)$$

Here, η_π and η_v are the learning rates for the policy and value networks, respectively.

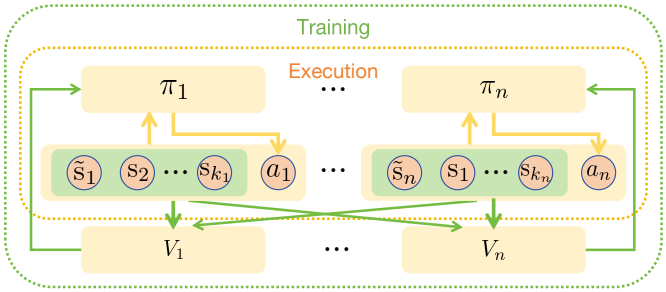


Fig. 3: Overview of the centralized training and decentralized execution.

C. Computational Complexity Analysis

The computational complexity of the temporal attention network is dominated by Eq. 17, with a complexity of $\Theta(T^3 + T^2 \cdot N)$, where T is the time window size and N is the number of servers. For the multi-graph convolution, the complexity primarily stems from Eq. 21, which involves a sparse-dense matrix multiplication of $\hat{A}_\tau H^{(l-1)}$, resulting in a complexity of $\Theta(|\mathcal{E}| \cdot F \cdot C)$. Here, $|\mathcal{E}|$ denotes the number of edges between servers (i.e., their correlations), while C and F represent the input and output feature dimensions, respectively. The actor network comprises m parallel fully connected subnetworks, each corresponding to a microservice, where m is the number of microservices. Each subnetwork consists of two fully connected layers that take the spatiotemporal information of the associated server as input and output an $|\mathcal{A}|$-dimensional action probability distribution, where $|\mathcal{A}|$ is the number of possible actions. Considering the parallel computation across m sub-networks, the total complexity of the actor network is $\Theta(d \cdot h + h \cdot |\mathcal{A}|)$, with d and h being the input dimension and hidden layer size, respectively.

TABLE II: System Parameters

Parameter	Description	Value
$\mathcal{W}_i^{cpu}$	Number of CPU cores	8–100
$\mathcal{W}_i^{mem}$	Memory size	16–64 GB
$\mathcal{W}_i^{disk}$	Storage volume	120–1024 GB
w_i^{cpu}	Required CPU core count	1–4
w_i^{mem}	Required memory size	0.5–4 GB
w_i^{disk}	Required storage volume	0.5–8 GB
$\text{B}_{\text{edge-edge}}(v_i, v_j)$	Bandwidth between edge nodes	1±0.2 Gbps
$\text{B}_{\text{edge-cloud}}(v_i, v_j)$	Bandwidth between edge and cloud	150±20 Mbps
λ_m	Processing rate of an instance	40–80 RPS
$\triangle_{m_i,m_j}$	The volume of transferred data	100–2000 KB

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

V. Evaluation

A. Experiment Setup

All experiments are conducted in a simulation environment using Python 3.9 on macOS equipped with an Apple M1 Pro chipset and 16 GB RAM. The system model parameters described in Section III-A and the algorithm-specific hyperparameters are summarized in Table II and Table III, respectively. Resource attributes (CPU, memory, and storage) are defined for edge servers, assuming cloud servers have unrestricted capacity. To avoid excessive autoscaling that may degrade QoS [61], the autoscaling interval is set to 30 seconds.

Workload and application. We construct geographically distributed workloads using the Shanghai Telecom trace [6]–[8], which contains over six months of Internet access records from 9,481 mobile users across 3,233 base stations, along with server location data derived from latitude and longitude. To simulate application requests, we assume that each user initiates one request per second during their active sessions, thereby converting access records into request traces. For the application scenario, we adopt a video processing (VP) service comprising 12 microservices, obtained from [62], with its architecture illustrated in Fig. 4.

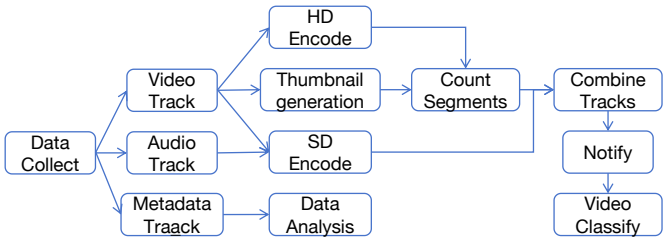


Fig. 4: Video Processing Application Architecture with 12 Microservices

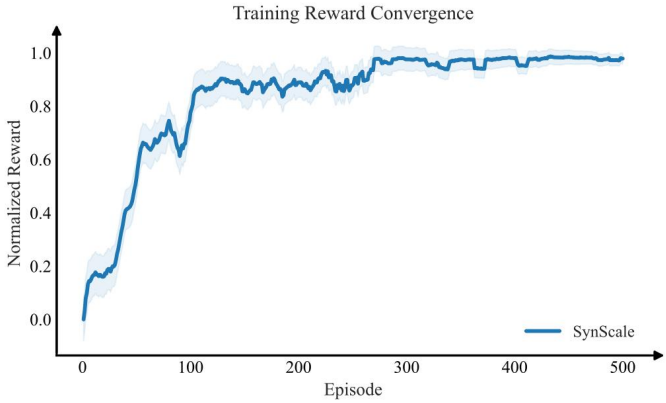


Fig. 5: Training reward convergence of SynScale.

Training Cost. We further report the training efficiency of SynScale. As shown in Fig. 5, the normalized training reward steadily increases and converges after approximately 500 episodes. The entire offline training phase for the eight-server setup completes within about 1 hours. Such training cost is acceptable since it is performed only once offline, while the subsequent inference process for autoscaling decisions incurs negligible latency. This result demonstrates that SynScale can be efficiently trained with modest computational resources while maintaining fast execution during deployment.

TABLE III: Algorithm Parameters

Parameter	Description	Value
c_1	Value loss coefficient	1
c_2	Entropy coefficient	0.01
ϵ	Clipping coefficient	0.2
γ	Discount factor	0.99
η_π	Learning rate for actor network	0.0005
η_v	Learning rate for critic network	0.0005

Comparison baselines. For comparison, we select three representative autoscaling algorithms: ProScale [9], HRA [40], and AWS algorithm [21]. These methods encompass a range of scaling strategies, from rule-based, prediction-driven approaches to adaptive learning models, providing a comprehensive baseline for evaluating our proposed framework.

- ProScale [9]: This predictive method adjusts the number of service instances based on predicted workloads using a heuristic algorithm. While ProScale offers a simple and intuitive approach, it relies on predefined rules and thresholds, making it inflexible to dynamic and heterogeneous environments. Moreover, it lacks coordination across servers, resulting in inefficient scaling decisions in distributed systems.
- HRA [40]: A reinforcement learning-based approach that enhances training efficiency by modeling environmental dynamics and utilizes DRL agents to make scaling decisions from low-dimensional features. While it exhibits improved learning performance compared to traditional DRL approaches, HRA still adopts a centralized policy structure and overlooks the distinct operational contexts of individual servers, leading to limited scalability and insufficient responsiveness in large-scale distributed systems.
- AWS algorithm [21]: A widely adopted threshold-based mechanism used in major platforms. In our experiments, we follow its standard configuration, where scaling decisions are triggered when CPU utilization exceeds a fixed threshold. Despite its industrial maturity and minimal computational overhead, AWS-style autoscaling lacks adaptability and fine-grained control, as it relies on static policies and narrow-scope indicators that are insufficient to respond to dynamic, heterogeneous runtime conditions.

Metrics. To evaluate the performance of the proposed framework, we adopt the following widely recognized metrics:

(1) Average Response Time (ART): The average end-to-end latency experienced by all requests during an evaluation episode, measured in milliseconds (ms):

$$\text{ART} = \frac{1}{N}\sum_{i=1}^{N} T_i, \tag{30}$$

where N is the total number of processed requests and T_i denotes the response time of the i-th request.

(2) SLA Violation Rate: In this work, we define a latency-centric service-level agreement (SLA), consistent with com-

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

mon practice in autoscaling literature [11], [40]. An SLA violation occurs when a request's response time exceeds a predefined threshold $\tau_{\text{SLA}} = 80\,\text{ms}$. The SLA violation rate is then defined as:

$$\text{SLA Violation Rate} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(T_i > \tau_{\text{SLA}}), \tag{31}$$

where $\mathbb{I}(\cdot)$ is the indicator function that returns 1 if the condition is true and 0 otherwise. This metric quantifies the proportion of requests failing to meet the latency-based SLA objective.

(3) 95th Percentile Latency (P95): To capture the tail latency that critically impacts user experience, we compute the 95th percentile of the response time distribution:

$$\text{P95} = \min\{t \mid F_T(t) \geq 0.95\}, \tag{32}$$

where $F_T(t)$ denotes the empirical cumulative distribution function (CDF) of request response times. A lower P95 value indicates tighter latency control under bursty workloads and reduced performance jitter.

(4) Resource Cost. We define the resource cost as the average number of active microservice instances across all servers over the evaluation period, reflecting the mean level of resource utilization. Formally,

$$\text{Cost} = \frac{1}{T}\sum_{t=1}^{T}\sum_{m\in M}\sum_{v\in V} n_{m,v}(t), \tag{33}$$

where $n_{m,v}(t)$ denotes the number of running instances of service m on server v at time slot t, and T is the total number of autoscaling intervals. A lower value of Cost indicates more efficient average resource usage while maintaining service-level performance.

(5) Instance-Time (IT). Since cloud providers typically charge according to the runtime of active instances, we quantify the total accumulated uptime of all microservice instances during an evaluation horizon T as

$$\text{IT} = \sum_{t=1}^{T}\sum_{m\in M}\sum_{v\in V} n_{m,v}(t)\cdot\Delta t, \tag{34}$$

where $n_{m,v}(t)$ denotes the number of running instances of service m on server v at time slot t, and Δt is the autoscaling interval (30 s in our experiments). The instance-time (IT) metric thus represents the total runtime accumulated by all instances, serving as a direct measure of temporal resource consumption. IT is reported in hours, where lower values indicate lower overall runtime cost.

(6) Average Instance Lifetime (AIL). While IT reflects the total uptime cost, it does not reveal how frequently instances are created and terminated. To assess scaling stability, we define the average instance lifetime as the mean duration that an instance remains active before being terminated. Formally, let ℓ_k denote the lifetime of the k-th instance, measured as the elapsed time between its start time $t_{\text{start}}^{(k)}$ and termination time $t_{\text{end}}^{(k)}$:

$$\ell_k = (t_{\text{end}}^{(k)} - t_{\text{start}}^{(k)})\cdot\Delta t. \tag{35}$$

The set of all observed lifetimes $\mathcal{L} = \{\ell_1, \ell_2, \ldots, \ell_K\}$ yields the following statistics:

$$\text{AIL} = \frac{1}{K}\sum_{\ell\in\mathcal{L}}\ell. \tag{36}$$

Here, AIL represents the average lifetime (reported in minutes). A higher AIL indicates that instances tend to stay active for longer periods, implying more stable and less oscillatory scaling with reduced cold-start overheads. Conversely, shorter lifetimes correspond to frequent scale-in/out cycles, reflecting more volatile behavior.

B. Experiment Results and Analysis

This section presents and analyzes our experimental results. To ensure reproducibility and statistical significance, each experiment was repeated 10 times, and we report the averaged results. All algorithms were trained under identical conditions, with the environment consisting of 8 servers, including 1 cloud server and 7 edge servers. To comprehensively evaluate our proposed method, we designed three sets of experiments, each addressing a specific validation aspect:

1) **Effectiveness validation:** Comparative analysis of the performance of different autoscaling algorithms.
2) **Ablation study:** Analysis of individual module contributions to the autoscaling performance in our approach.
3) **Overhead analysis:** Comparison of overhead across different algorithms under environments of varying scales.

Note that the experimental setups for the first two experiments were conducted with environments of the same scale as the training environment.

Effectiveness validation. This experiment evaluates the overall effectiveness of SynScale under dynamic, heterogeneous workloads. As shown in Fig. 6(a), SynScale achieves the lowest and most stable response time throughout the autoscaling process. The kernel density distribution in Fig. 6(c) further confirms its tight latency profile, with most requests concentrated in the low-latency region and significantly fewer long-tail outliers. Across all metrics (Fig. 6(b)), SynScale consistently outperforms state-of-the-art methods, reducing average response time by 59.32%, SLA violations by 85.71%, and P95 latency by 73.2%, while lowering resource cost by 14.46%. It also achieves 42.77% lower instance-time and 54.5% longer instance lifetime, indicating more stable scaling with fewer oscillatory scale-in/out cycles.

The superiority of SynScale stems from its ability to couple temporal consistency with structure-aware coordination. The temporal module captures how past actions influence current performance, enabling trend-aligned scaling rather than short-sighted reactions. Meanwhile, the spatial module leverages explicitly modeled inter-server synergies to form structure-aware representations, allowing each agent to make locally adaptive yet globally coherent decisions. This joint spatiotemporal awareness enables SynScale to provision resources proactively where needed while avoiding redundant scaling across the system, leading to both improved QoS and reduced cost.

Compared with SynScale, existing methods reveal clear structural limitations. Rule-based autoscalers—typified by systems such as AWS—depend on statically configured thresholds

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

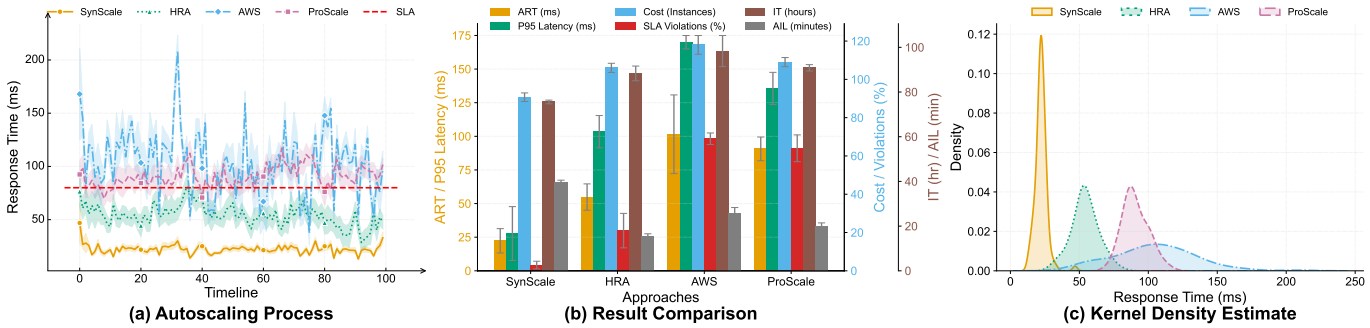


Fig. 6: Performance Comparison Across Different Algorithms

TABLE IV: Ablation Study Results

Metric	Full Model (Exp. 1)	w/o Temporal (Exp. 2)	w/o Spatial (Exp. 3)	w/o Spatiotemporal (Exp. 4)
ART (ms)	$\mathbf{22.3 \pm 6.28}$	35.32 ± 4.31	42.33 ± 10.32	48.08 ± 12.48
P95 Latency (ms)	$\mathbf{27.72 \pm 2.13}$	82.4 ± 14	70.28 ± 19	91.22 ± 6.29
Cost (Instances)	$\mathbf{90.67 \pm 4.2}$	99.82 ± 1.37	106.18 ± 4.22	128.34 ± 7.6
SLA Violations (%)	$\mathbf{2.2 \pm 0.9}$	11.3 ± 2.9	13.6 ± 5.6	18 ± 3.8
IT (hours)	$\mathbf{75.56 \pm 0.83}$	80.28 ± 2.05	91.38 ± 1.1	104.89 ± 0.98
AIL (minutes)	$\mathbf{39.72 \pm 0.53}$	35.5 ± 0.9	37.02 ± 1.04	29.93 ± 0.87

and narrow-scope indicators, which cannot reflect evolving workload trends or cross-server interactions. This rigidity leads to delayed or excessive scaling actions and consequently higher latency and violation rates under dynamic conditions. Prediction-driven approaches (e.g., ProScale) improve reactivity but rely on one-dimensional forecasts and predefined heuristics, making them sensitive to prediction errors and unstable when workload patterns shift. Centralized RL methods such as HRA are more adaptive yet still encode the environment through a monolithic global policy, limiting their ability to capture server-specific heterogeneity and hindering distributed coordination. These design constraints fundamentally restrict their capability to sustain low latency and stable scaling in heterogeneous edge-cloud settings.

Ablation study. To assess the contribution of each core component in SynScale, we conducted a series of ablation experiments, with results reported in Table IV. The complete model consistently outperforms all ablated variants across key performance metrics, underscoring the importance of each module.

Experiment 2 removes the temporal-awareness module, which leverages historical observations to uncover the causal impact of recent scaling actions on system performance. Without this capability, the system cannot accurately model the delayed effects of prior decisions, leading to less informed policies and degraded responsiveness—particularly evident in elevated P95 latency, which reflects poor temporal stability.

Experiment 3 disables the spatial module, which implements structured representation learning over explicitly modeled server relationships using graph convolution. This module enables context-aware refinement of server states by propagating relational information across distributed nodes. Its removal undermines cross-server awareness and collaboration, leading to inefficient resource utilization and increased SLA violations.

Experiment 4 removes both the spatial and temporal modules. This combined ablation yields the poorest performance, confirming that relationally informed spatiotemporal modeling is central to SynScale's ability to achieve effective collaborative autoscaling under dynamic and heterogeneous conditions.

Overhead analysis. In edge-cloud scenarios, where edge devices often operate under tight latency and resource constraints, the runtime overhead introduced by autoscaling strategies is a critical factor for practical deployment. We evaluate the overhead of each method in terms of the time (ms) and memory usage (MB) required to complete a one-shot scaling decision under environments of varying scale (i.e., different numbers of servers). Since the goal of this experiment is to measure inference-time overhead rather than policy performance, we directly execute the trained model on environments with different numbers of servers without retraining. The forward computation—including the multi-graph convolution in Eq. (23) and the shared actor network—is fully compatible with varying environment sizes, making this setup appropriate for isolating the impact of system scale on computational cost. The results are summarized in Table V.

As system size increases, most methods show rising overhead due to the need to process more complex system information. AWS, which employs a simple threshold-based trigger without structural or historical reasoning, maintains low overhead but lacks adaptability and struggles to meet QoS under dynamic workloads. ProScale incurs significantly higher overhead because it must evaluate heuristic rules over both the service dependency graph and all candidate servers when making scaling decisions. HRA, relying on a centralized RL formulation, performs policy inference over a high-dimensional global state, causing inference latency and memory usage to grow rapidly as the server count increases.

In contrast, SynScale maintains a lightweight and slowly

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

TABLE V: Overhead Comparison

Exp. ID	Scale	Metric	SynScale	ProScale	AWS	HRA
1	8	Exec. Time (ms)	2.85 ± 1.02	9.44 ± 3.62	2.21 ± 1.29	12.95 ± 9.13
		Memory (MB)	0.50 ± 1.34	0.27 ± 0.42	0.40 ± 0.96	0.63 ± 1.40
2	15	Exec. Time (ms)	3.64 ± 0.97	13.27 ± 0.70	2.86 ± 0.28	17.91 ± 0.61
		Memory (MB)	0.62 ± 1.36	0.27 ± 0.48	0.49 ± 0.97	0.65 ± 1.45
3	50	Exec. Time (ms)	$\mathbf{13.59 \pm 8.42}$	41.42 ± 2.40	14.49 ± 1.33	50.70 ± 1.39
		Memory (MB)	1.21 ± 1.82	0.56 ± 0.77	1.00 ± 1.36	0.93 ± 1.65
4	70	Exec. Time (ms)	$\mathbf{17.26 \pm 0.86}$	57.27 ± 3.26	17.71 ± 1.76	70.73 ± 1.36
		Memory (MB)	3.04 ± 4.51	0.53 ± 1.21	1.06 ± 1.46	1.26 ± 1.89
5	100	Exec. Time (ms)	32.21 ± 1.75	82.36 ± 6.72	26.78 ± 3.63	102.72 ± 2.34
		Memory (MB)	7.52 ± 9.22	1.06 ± 1.40	0.99 ± 1.74	1.65 ± 2.08

increasing overhead. Its distributed design ensures that each agent reasons over localized information rather than a global system state, yielding nearly linear inference cost—2.85 ms $\rightarrow$ 32.21 ms from 8 to 100 servers ($\approx$0.32 ms/server), compared with HRA ($\approx$0.98 ms/server) and ProScale ($\approx$0.79 ms/server). The slight upward trend originates from computing graph-based relational representations via the multi-graph convolution in Eq. 21. However, the correlation matrices are typically sparse—enabling efficient sparse operations (§IV-C)—and the aggregation neighborhood can be flexibly constrained, so the receptive field does not have to grow with the full system size. These factors keep SynScale's scale-related overhead moderate and predictable, offering a favorable overhead-performance balance suitable for large-scale, latency-sensitive deployments.

VI. Discussion and Limitations

Feasibility discussion. Our overhead evaluation shows that the proposed method incurs minimal delay and resource consumption, exerting negligible impact on autoscaling performance. The framework supports both distributed and centralized deployment, enabling flexible integration across heterogeneous environments. When edge servers have sufficient capacity, local deployment allows parallel, server-side decision-making. In resource-limited scenarios, the policy can be centrally executed on a cloud server, with node-level metrics collected through high-performance monitoring systems (e.g., Prometheus [63]). Although centralized deployment introduces synchronization overhead, its magnitude is largely determined by the monitoring infrastructure, and modern tools generally keep this overhead within acceptable bounds.

Correlation modeling limitation. A current limitation of our framework lies in its reliance on manually constructed inter-server relationships to represent collaborative synergies. While effective in our setting, such predefined structures may miss latent or evolving dependencies, especially in large, dynamic, or previously unseen environments. Moreover, new deployment scenarios may require additional effort to adapt or redesign these relational models, introducing manual tuning. To mitigate this limitation, future work will explore automated approaches for inferring inter-server relationships, such as unsupervised or self-organizing techniques.

Transferability limitation. We adopt parameter sharing across agents, so the learned policy can be directly executed in environments with different numbers of servers. However, transfer becomes limited because the structural features in SynScale are extracted using a spectral GCN, whose filters depend on the eigenstructure of the training-time graphs and therefore exhibit limited transfer when the topology or server relationships differ substantially. In addition, the policy network is tied to a fixed per-server state space, so any change in the state definition would require redesigning the input representation and retraining the model, although such state definitions are typically stable in real deployments. Environment-level transferability remains a broader challenge for autoscaling systems operating across heterogeneous environments, and we plan to design autoscaling algorithms that better support such environment transfer in future work.

VII. Conclusion

This paper presented SynScale, a distributed collaborative autoscaling framework for microservices in heterogeneous edge-cloud environments. By explicitly modeling inter-server relationships and integrating spatiotemporal representation learning with multi-agent reinforcement learning, SynScale enables locally adaptive yet globally coordinated scaling decisions across distributed servers. Experimental results demonstrate that SynScale consistently outperforms existing autoscaling approaches in terms of response latency, SLA compliance, and resource efficiency. Moreover, the framework maintains stable and lightweight runtime overhead across different system scales, indicating its practicality for large-scale deployments. These results highlight that spatiotemporal correlation-aware and distributed scaling paradigms can provide a principled foundation for autoscaling systems operating in large-scale edge-cloud infrastructures.

VIII. Acknowledgements

This work is Supported by the National Natural Science Foundation of China(NSFC) under Grant No.92582202, and the NSFC-Guangdong Joint Fund Project under Grant No.U20A6003.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

REFERENCES

[1] Y. Chiang, Y. Zhang, H. Luo, T.-Y. Chen, G.-H. Chen, H.-T. Chen, Y.-J. Wang, H.-Y. Wei, and C.-T. Chou, "Management and orchestration of edge computing for iot: A comprehensive survey," *IEEE Internet of Things Journal*, 2023.

[2] A. Sarah, G. Nencioni, and M. M. I. Khan, "Resource allocation in multi-access edge computing for 5g-and-beyond networks," *Computer Networks*, vol. 227, p. 109720, 2023.

[3] "CNCF". (2021). [Online]. Available: https://www.cncf.io/

[4] AWS, "Hybrid cloud with aws," 2022, [Online]. [Online]. Available: https://aws.amazon.com/hybrid/

[5] Google, "Building enterprise edge applications with google cloud," 2022, [Online]. [Online]. Available: https://cloud.google.com/blog/topics/hybrid-cloud/google-cloud-tools-to-manage-your-enterprise-edge-apps

[6] Y. Guo, S. Wang, A. Zhou, J. Xu, J. Yuan, and C.-H. Hsu, "User allocation-aware edge cloud placement in mobile edge computing," *Software: Practice and Experience*, vol. 50, no. 5, pp. 489–502, 2020.

[7] S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. Shen, "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Transactions on Mobile Computing*, vol. 20, no. 3, pp. 939–951, 2019.

[8] Y. Li, A. Zhou, X. Ma, and S. Wang, "Profit-aware edge server placement," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 55–67, 2021.

[9] K. Cheng, S. Zhang, C. Tu, X. Shi, Z. Yin, S. Lu, Y. Liang, and Q. Gu, "Proscale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Transactions on Parallel and Distributed Systems*, 2023.

[10] C. Meng, S. Song, H. Tong, M. Pan, and Y. Yu, "Deepscaler: Holistic autoscaling for microservices based on spatiotemporal gnn with adaptive graph learning," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 53–65.

[11] S. Zhang, T. Wu, M. Pan, C. Zhang, and Y. Yu, "A-sarsa: A predictive container auto-scaling algorithm based on reinforcement learning," in *2020 IEEE International Conference on Web Services (ICWS)*. IEEE, 2020, pp. 489–497.

[12] K. Cheng, S. Zhang, M. Liu, Y. Gu, L. Wei, H. Cheng, K. Liu, Y. Song, X. Shi, A. Zhu *et al.*, "Geoscale: Microservice autoscaling with cost budget in geo-distributed edge clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 4, pp. 646–662, 2024.

[13] X. He, Z. Tu, M. Wagner, X. Xu, and Z. Wang, "Online deployment algorithms for microservice systems with complex dependencies," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1746–1763, 2022.

[14] J. Tong, M. Wei, M. Pan, and Y. Yu, "A holistic auto-scaling algorithm for multi-service applications based on balanced queuing network," in *2021 IEEE International Conference on Web Services (ICWS)*. IEEE, 2021, pp. 531–540.

[15] W. Lv, P. Yang, T. Zheng, C. Lin, Z. Wang, M. Deng, and Q. Wang, "Graph reinforcement learning-based dependency-aware microservice deployment in edge computing," *IEEE Internet of Things Journal*, 2023.

[16] C. Wang, B. Jia, H. Yu, X. Li, X. Wang, and T. Taleb, "Deep reinforcement learning for dependency-aware microservice deployment in edge computing," in *GLOBECOM 2022-2022 IEEE Global Communications Conference*. IEEE, 2022, pp. 5141–5146.

[17] K. Fu, W. Zhang, Q. Chen, D. Zeng, X. Peng, W. Zheng, and M. Guo, "Qos-aware and resource efficient microservice deployment in cloud-edge continuum," in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2021, pp. 932–941.

[18] L. Chen, Y. Xu, Z. Lu, J. Wu, K. Gai, P. C. Hung, and M. Qiu, "Iot microservice deployment in edge-cloud hybrid environment using reinforcement learning," *IEEE Internet of Things Journal*, vol. 8, no. 16, pp. 12 610–12 622, 2020.

[19] L. A. Vayghan, M. A. Saied, M. Toeroe, and F. Khendek, "A kubernetes controller for managing the availability of elastic microservice based stateful applications," *Journal of Systems and Software*, vol. 175, p. 110924, 2021.

[20] S. N. Srirama, M. Adhikari, and S. Paul, "Application deployment using containers with auto-scaling for microservices in cloud environment," *Journal of Network and Computer Applications*, vol. 160, p. 102629, 2020.

[21] (2025) Amazon Auto Scaling Service. Accessed: Aug 10, 2025. [Online]. Available: http://aws.amazon.com/autoscaling/

[22] W. Ma, R. Wang, Y. Gu, Q. Meng, H. Huang, S. Deng, and Y. Wu, "Multi-objective microservice deployment optimization via a knowledge-driven evolutionary algorithm," *Complex & Intelligent Systems*, vol. 7, pp. 1153–1171, 2021.

[23] S. Xie, J. Wang, B. Li, Z. Zhang, D. Li, and P. C. Hung, "Pbscaler: A bottleneck-aware autoscaling framework for microservice-based applications," *IEEE Transactions on Services Computing*, 2024.

[24] D. Balla, C. Simon, and M. Maliosz, "Adaptive scaling of kubernetes pods," in *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2020, pp. 1–5.

[25] E. Casalicchio and V. Perciballi, "Auto-scaling of containers: The impact of relative and absolute metrics," in *2017 IEEE 2nd International Workshops on Foundations and Applications of Self* Systems (FAS* W)*. IEEE, 2017, pp. 207–214.

[26] M. R. Hossen and M. A. Islam, "A lightweight workload-aware microservices autoscaling with qos assurance," *learning*, vol. 12, pp. 17–19, 2022.

[27] L. Baresi and G. Quattrocchi, "A simulation-based comparison between industrial autoscaling solutions and cocos for cloud applications," in *2020 IEEE International Conference on Web Services (ICWS)*. IEEE, 2020, pp. 94–101.

[28] A. F. Baarzi and G. Kesidis, "Showar: Right-sizing and efficient scheduling of microservices," in *Proceedings of the ACM Symposium on Cloud Computing*, 2021, pp. 427–441.

[29] M. Mekki, N. Toumi, and A. Ksentini, "Microservices configurations and the impact on the performance in cloud native environments," in *2022 IEEE 47th Conference on Local Computer Networks (LCN)*. IEEE, 2022, pp. 239–244.

[30] Y. Guo, J. Ge, P. Guo, Y. Chai, T. Li, M. Shi, Y. Tu, and J. Ouyang, "Pass: Predictive auto-scaling system for large-scale enterprise web applications," in *Proceedings of the ACM on Web Conference 2024*, 2024, pp. 2747–2758.

[31] H. Yuan and S. Liao, "A time series-based approach to elastic kubernetes scaling," *Electronics*, vol. 13, no. 2, p. 285, 2024.

[32] S. K. Mondal, X. Wu, H. M. D. Kabir, H.-N. Dai, K. Ni, H. Yuan, and T. Wang, "Toward optimal load prediction and customizable autoscaling scheme for kubernetes," *Mathematics*, vol. 11, no. 12, p. 2675, 2023.

[33] L. Ju, P. Singh, and S. Toor, "Proactive autoscaling for edge computing systems with kubernetes," in *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing Companion*, 2021, pp. 1–8.

[34] Y. Song, C. Li, K. Zhuang, T. Ma, and T. Wo, "An automatic scaling system for online application with microservices architecture," in *2022 IEEE International Conference on Joint Cloud Computing (JCC)*. IEEE, 2022, pp. 73–78.

[35] Y. Li, H. Zhang, W. Tian, and H. Ma, "Joint optimization of auto-scaling and adaptive service placement in edge computing," in *2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2021, pp. 923–930.

[36] Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou, "Sinan: Ml-based and qos-aware resource management for cloud microservices," in *Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems*, 2021, pp. 167–181.

[37] X. Li, J. Zhou, X. Wei, D. Li, Z. Qian, J. Wu, X. Qin, and S. Lu, "Topology-aware scheduling framework for microservice applications in cloud," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1635–1649, 2023.

[38] G. Yu, P. Chen, and Z. Zheng, "Microscaler: Cost-effective scaling for microservice applications in the cloud with an online learning approach," *IEEE Transactions on Cloud Computing*, vol. 10, no. 2, pp. 1100–1116, 2020.

[39] H. Yang, Q. Chen, M. Riaz, Z. Luan, L. Tang, and J. Mars, "Powerchief: Intelligent power allocation for multi-stage applications to improve responsiveness on power constrained cmp," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 133–146.

[40] C. Meng, J. Tong, M. Pan, and Y. Yu, "Hra: An intelligent holistic resource autoscaling framework for multi-service applications," in *2022 IEEE International Conference on Web Services (ICWS)*. IEEE, 2022, pp. 129–139.

[41] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo, "Adaptive resource efficient microservice deployment in cloud-edge continuum," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1825–1840, 2021.

[42] Z. Yang, P. Nguyen, H. Jin, and K. Nahrstedt, "Miras: Model-based reinforcement learning for microservice resource allocation over scientific workflows," in *2019 IEEE 39th international conference on distributed computing systems (ICDCS)*. IEEE, 2019, pp. 122–132.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3672566

[43] W. Lv, Q. Wang, P. Yang, Y. Ding, B. Yi, Z. Wang, and C. Lin, "Microservice deployment in edge computing based on deep q learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2968–2978, 2022.

[44] D. Garg, N. C. Narendra, and S. Tesfatsion, "Heuristic and reinforcement learning algorithms for dynamic service placement on mobile edge cloud," *arXiv preprint arXiv:2111.00240*, 2021.

[45] X. Tian, H. Meng, Y. Shen, J. Zhang, Y. Chen, and Y. Li, "Dynamic microservice deployment and offloading for things-edge-cloud computing," *IEEE Internet of Things Journal*, 2024.

[46] K. Gasmi, K. Abassi, L. Romdhani, and O. Debauche, "Reinforcement learning-based approach for microservices-based application placement in edge environment," in *2023 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2023, pp. 1098–1103.

[47] S. Chen, Q. Yuan, J. Li, H. He, S. Li, X. Jiang, and J. Yang, "Graph neural network aided deep reinforcement learning for microservice deployment in cooperative edge computing," *IEEE Transactions on Services Computing*, 2024.

[48] S. Horovitz and Y. Arian, "Efficient cloud auto-scaling with sla objective using q-learning," in *2018 IEEE 6th International Conference on Future Internet of Things and Cloud (FiCloud)*. IEEE, 2018, pp. 85–92.

[49] G. Yu, P. Chen, and Z. Zheng, "Microscaler: Automatic scaling for microservices with an online learning approach," in *2019 IEEE International Conference on Web Services (ICWS)*. IEEE, 2019, pp. 68–75.

[50] X. Jie, T. Liu, H. Gao, C. Cao, P. Wang, and W. Tong, "A dqn-based approach for online service placement in mobile edge computing," in *Collaborative Computing: Networking, Applications and Worksharing: 16th EAI International Conference, CollaborateCom 2020, Shanghai, China, October 16–18, 2020, Proceedings, Part II 16*. Springer, 2021, pp. 169–183.

[51] T.-T. Nguyen, Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, "Horizontal pod autoscaling in kubernetes for elastic container orchestration," *Sensors*, vol. 20, no. 16, p. 4621, 2020.

[52] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE internet of things journal*, vol. 3, no. 5, pp. 637–646, 2016.

[53] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE communications surveys & tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[54] S. Deng, Z. Xiang, J. Taheri, M. A. Khoshkholghi, J. Yin, A. Y. Zomaya, and S. Dustdar, "Optimal application deployment in resource constrained distributed edges," *IEEE transactions on mobile computing*, vol. 20, no. 5, pp. 1907–1923, 2020.

[55] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," *Advances in neural information processing systems*, vol. 29, 2016.

[56] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 611–24 624, 2022.

[57] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," *Advances in neural information processing systems*, vol. 30, 2017.

[58] F. Christianos, G. Papoudakis, M. A. Rahman, and S. V. Albrecht, "Scaling multi-agent reinforcement learning with selective parameter sharing," in *International Conference on Machine Learning*. PMLR, 2021, pp. 1989–1998.

[59] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[60] H. P. Van Hasselt, A. Guez, M. Hessel, V. Mnih, and D. Silver, "Learning values across many orders of magnitude," *Advances in neural information processing systems*, vol. 29, 2016.

[61] M. Abdullah, W. Iqbal, J. L. Berral, J. Polo, and D. Carrera, "Burst-aware predictive autoscaling for containerized microservices," *IEEE Transactions on Services Computing*, vol. 15, no. 3, pp. 1448–1460, 2020.

[62] Q. Huang, P. Ang, P. Knowles, T. Nykiel, I. Tverdokhlib, A. Yajurvedi, P. Dapolito IV, X. Yan, M. Bykov, C. Liang *et al.*, "Sve: Distributed video processing at facebook scale," in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 87–103.

[63] Prometheus, "An open-source monitoring system with a dimensional data model, flexible query language, efficient time series database and modern alerting approach." 2025. [Online]. Available: https://prometheus.io/



Haogang Tong received his M.S. degree from Beijing University of Posts and Telecommunications (BUPT), China, in 2020. He is pursuing a Ph.D. degree with the School of Computer Science, Sun Yat-sen University (SYSU), China. His research focuses on microservices scheduling in distributed systems, latency-sensitive task orchestration for edge computing, and IoT-driven intelligent systems. His work was awarded the ACM SIGSOFT Distinguished Paper Award.



Chunyang Meng received his master degree from Beijing University of Posts and Telecommunications, China, in 2020. He is now a Ph.D. student at School of Computer Science and Engineering with Sun Yat-Sen University, China. His current research areas include distributed system, services computing, and AI driven operations. He was a recipient of the ACM SIGSOFT Distinguished Paper Award at ASE 2023 and the Best Student Paper Award at ICWS 2022.



Shijie Song received his bachelor's degree from Jinan University, China, in 2022. He is currently a master's student at the School of Computer Science and Engineering, Sun Yat-Sen University, China. His current research areas include distributed systems, services computing, and serverless functions. He was a recipient of the ACM SIGSOFT Distinguished Paper Award at ASE 2023 and has a paper accepted at ICWS 2024.



Maolin Pan received the B.S. degree from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 1988, and the M.S. degree from HUST, in 1991, and the Ph.D. degree from Sun Yat-sen University (SYSU), Guangzhou, China, in 2017. He is currently a lecturer with the School of Software Engineering, SYSU. His research interests include BPM and workflow technical, cloud native computing.



Yang Yu received the bachelor's and master's degrees in computer science from Huazhong University of Science and Technology (HUST), Wuhan, China, and the Ph.D. degree in computer science from Sun Yat-sen University (SYSU), Guangzhou, China, in 1988, 1991, and 2007, respectively. He is currently a Full Professor with the School of Data and Computer Science and the School of Software Engineering, Sun Yat-sen University. His research interests include workflow/BPM, service computing, cloud computing, and software engineering. He is a Distinguished Member of CCF and a member of ACM.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:08:26 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.